

UNIVERSIDAD ESTATAL A DISTANCIA
VICERRECTORIA ACADEMICA
ESCUELA DE CIENCIAS EXACTAS Y NATURALES
CARRERA INGENIERÍA INFORMATICA

PROYECTO No 3

MODALIDAD ESCOGIDA: PROYECTO

Grupo: 02

PROYECTO #3 PARA EL CURSO
DE ANALISIS Y DISEÑO DE SISTEMAS

PABLO ANDRÉ VALENCIANO BLANCO

1-1572-0043

Profesor: Jorge Alvarado Zamora

CENTRO UNIVERSITARIO DE HERERIA

PAC: 2025-2

Fecha Entrega: 27 de Julio 2025

CIUDAD: HEREDIA

Contenido

INTRODUCCIÓN	1
Parte 1. PATRONES ARQUITECTONICOS	2
Sección A. Descripción de cada modelo	2
Sección B. Imagen para cada modelo	3
Sección C. El mejor modelo para RedBiln	5
Parte 2. DIAGRAMA DE DESPLIEGUE PARA EL DISEÑO DE REDBIIN	6
Parte 3. MODELADO DE HISTORIAS DEL USUARIO PARA EL CASO DE ESTUDIO	7
Parte 4. STORY MAPPING	8
Parte 5. RIESGOS ASOCIADOS A LOS REQUERIMIENTOS DEL PROYECTO	9
Parte 6. PRINCIPIOS SOLID	10
Sección A. Definición	10
Sección B. Explicación de cada uno de los principios	10
Parte 7. ESTANDAR DE CALIDAD	12
Sección A. ISO/IEC 25010:2023 – Modelo de calidad del producto software (SQuaRE), explicación	12
Sección B. Características de dicho estándar de calidad	12
CONCLUSIÓN	13
REFERENCIAS	14

Tabla de Figuras

Figura 1	3
Figura 2	3
Figura 3	4
Figura 4	4
Figura 5	6
Figura 6	8

Tabla de Tablas

Tabla 1	2
Tabla 2	7
Tabla 3	9

INTRODUCCIÓN

Finalmente, se ha llegado al epílogo de los proyectos para el curso de Análisis de Sistemas, siendo este el tercer y último proyecto del curso, el cual a través de la materia se han visto varios temas que rodean a un pre-análisis y una preparación adecuada para poder realizar el ajuste a la solución del problema propuesto. Y este proyecto no es una excepción a esto, sino más bien a complementar los aprendizajes previos con nuevas visiones y alternativas que según el entorno del proyecto se pueda abarcar de una forma u otra.

El objetivo de esta actividad según menciona el mismo proyecto (2025) es que “el estudiante aplicará las técnicas estudiadas en la literatura del curso aplicables al diseño arquitectónico de software, profundización en técnicas de implementación de código que cumpla con los principios SOLID, y estándares de calidad del software.” Los cuales se verán con más detalle al consultar cada una de las partes que conforman el documento actual.

La presentación del mismo, será similar al de los proyectos anteriores y siguiendo las estructuras para referencia APA séptima edición y los lineamientos para trabajos escritos en la carrera de Informática, siguiendo las instrucciones del proyecto de manera concisa y al punto y siempre cuidando los detalles de la presentación.

Las partes que van a estar incluidas en el proyecto actual en orden de aparición en el cual se verán temas como los patrones arquitectónicos, el cual se subdividirá en sus definiciones y características que cada uno de esta posee y cual se adecua más al Caso de Estudio RedBiln, luego se verá un diagrama de despliegue que incluirá gran parte de las funcionalidades de la propuesta de solución, se afinarán las historias de usuario a través de un modelado de tabla, se explica lo que es el “story mapping” apoyándose en ilustraciones, se piensa hacer una búsqueda de 5 posibles riesgos que puedan suceder a lo largo del proyecto y dar una explicación sobre cada uno de ellos, por última se mencionará los principios y las características de cada uno de estos en búsqueda de los estándares de calidad y como estos apoyan a la creación para métricas, que serán la vara de medición en busca de satisfacer las necesidades del caso de estudio.

Parte 1. PATRONES ARQUITECTONICOS

Sección A. Descripción de cada modelo

Patrón Arquitectónico	Descripción	Ventajas	Desventajas
Modelo-Vista-Controlador	Divide la aplicación en tres componentes: el Modelo (lógica de negocio y datos), la Vista (interfaz de usuario) y el Controlador (gestión de eventos y coordinación)	1. Separación de responsabilidades, lo que mejora el mantenimiento y permite que equipos trabajen en paralelo. 2. Reutilización de código y escalabilidad, se pueden reutilizar modelos o vistas en diferentes contextos.	1. Complejidad adicional, especialmente en proyectos pequeños, y curva de aprendizaje para principiantes. 2. Código “boilerplate” excesivo: necesidad de crear tres componentes por cada funcionalidad, lo que ralentiza el desarrollo
Arquitectura en Capas	Organiza el sistema en capas jerárquicas (presentación, lógica de negocio, acceso a datos, etc.), donde cada capa solo interactúa con la capa inmediata superior/inferior.	1. Modularidad y mantenibilidad, cada capa es independiente y facilita la gestión del cambio. 2. Facilita pruebas y reutilización, ya que las capas pueden probarse aisladamente y reutilizarse.	1. Rendimiento: puede generar sobrecarga por múltiples llamadas entre capas. 2. Interacción rígida entre capas, dificulta optimizaciones que cruzan capas.
Arquitectura Cliente-Servidor	Modelo distribuido que separa los componentes en clientes (interfaz, experiencia de usuario) y servidores (procesamiento, almacenamiento, servicios)	1. Gestión centralizada, servidores controlan datos, actualizaciones y seguridad. 2. Escalabilidad modular, se agregan o mejoran servidores sin afectar clientes	1. Punto único de falla, si el servidor cae, todos los clientes quedan sin servicio. 2. Costos y complejidad: requiere inversión en hardware, administración, y redes robustas

Tabla 1. Patrones Arquitectónicos

Sección B. Imagen para cada modelo

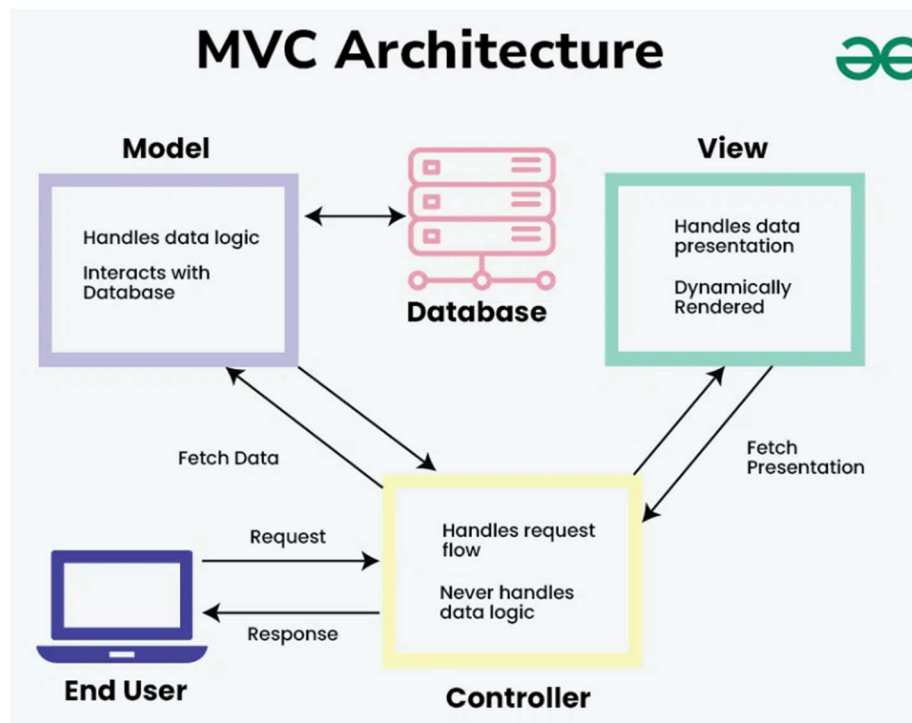


Figura 1. Patrón Arquitectónico: MVC

Imagen propiedad de geeksforgeeks (2024)

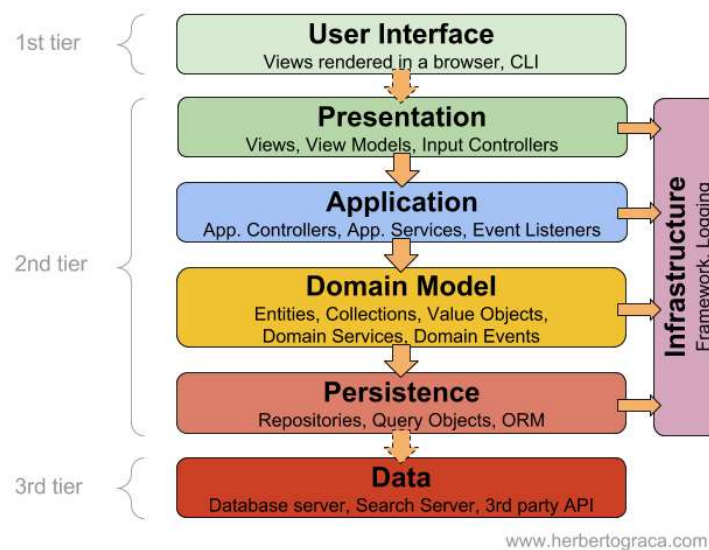


Figura 2. Patrón Arquitectónico: Arquitectura de capas

Imagen propiedad de herbertograca (2017)

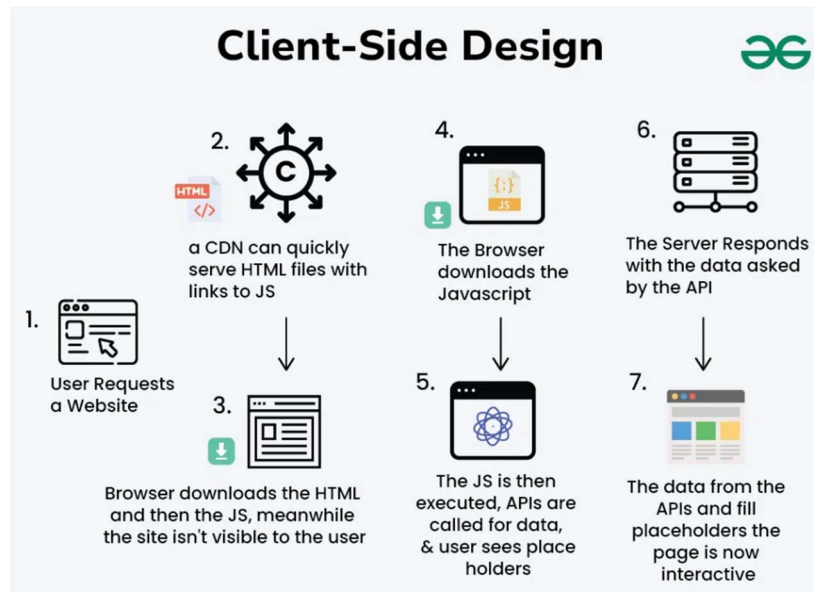


Figura 3. Patrón Arquitectónico: Arquitectura Cliente-Servidor, lado Cliente
 Imagen propiedad de geeksforgeeks (2024)

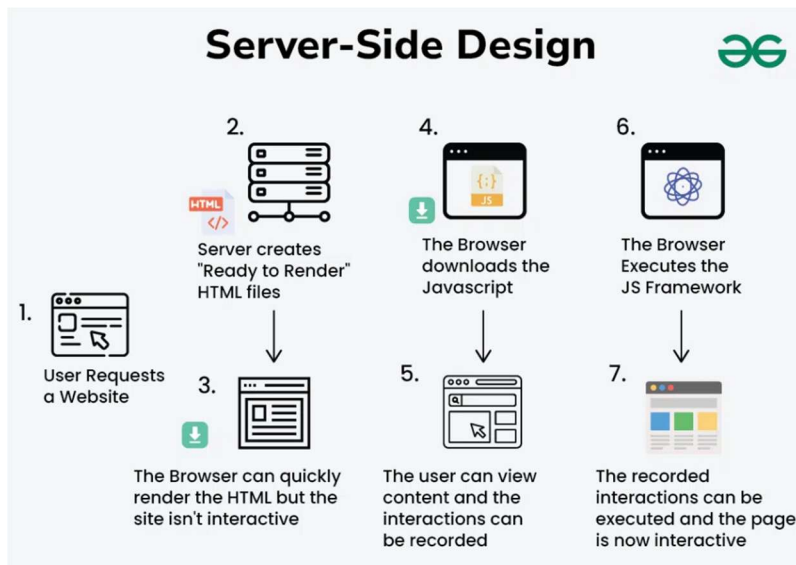


Figura 4. Patrón Arquitectónico: Arquitectura Cliente-Servidor, lado Servidor
 Imagen propiedad de geeksforgeeks (2024)

Sección C. El mejor modelo para RedBiln

Respuesta rápida: la mejor arquitectura y más compatible con el caso de estudio es la del Cliente-Servidor. Ahora la explicación del por qué esta sobre las otras dos, es cuestión de ciertas características que la hacen mejor. Entre los beneficios que la arquitectura Cliente-Servidor provee al caso de estudio pueden ser:

- Permite centralizar datos y servicios, asegurando control, integridad y seguridad del catálogo y préstamos.
- Escalable: se puede expandir añadiendo servidores, replicando base de datos o servicios.
- Los costos suben, cuando se integran más bibliotecas y no más clientes o tipos de clientes, siendo escalable y no necesita crear código nuevo, ya que el servidor es el que administra las funcionalidades y el cliente solo ve lo que el servidor manda.

Mientras que el modelado MVC, como alternativa es estable y constante posee la mayor problemática que si hay algún cambio en cualquiera de las tres capas, afecta todo lo involucrado al atributo o clase asociado. Creando la necesidad de mandar actualizaciones a clientes sin cambios reales o funcionales para ellos.

Y la arquitectura de capas, ofrece demasiada complejidad a elementos que ya en físico se hacen de manera simple, como lo es administrar los libros, pagos de préstamos y avisos por mensajes. También, las clases que se necesitan no son tan dependientes de otras para que se puedan realizar, de modo que los sistemas en cierto modo, solo requieren saber el actor, quien está ejecutando y el procedimiento.

La solución arquitectónica Cliente-Servidor, aunque es la más cara provee mayor satisfacción al cliente y su uso es más duradero a lo largo de años.

Parte 2. DIAGRAMA DE DESPLIEGUE PARA EL DISEÑO DE REDBIIN

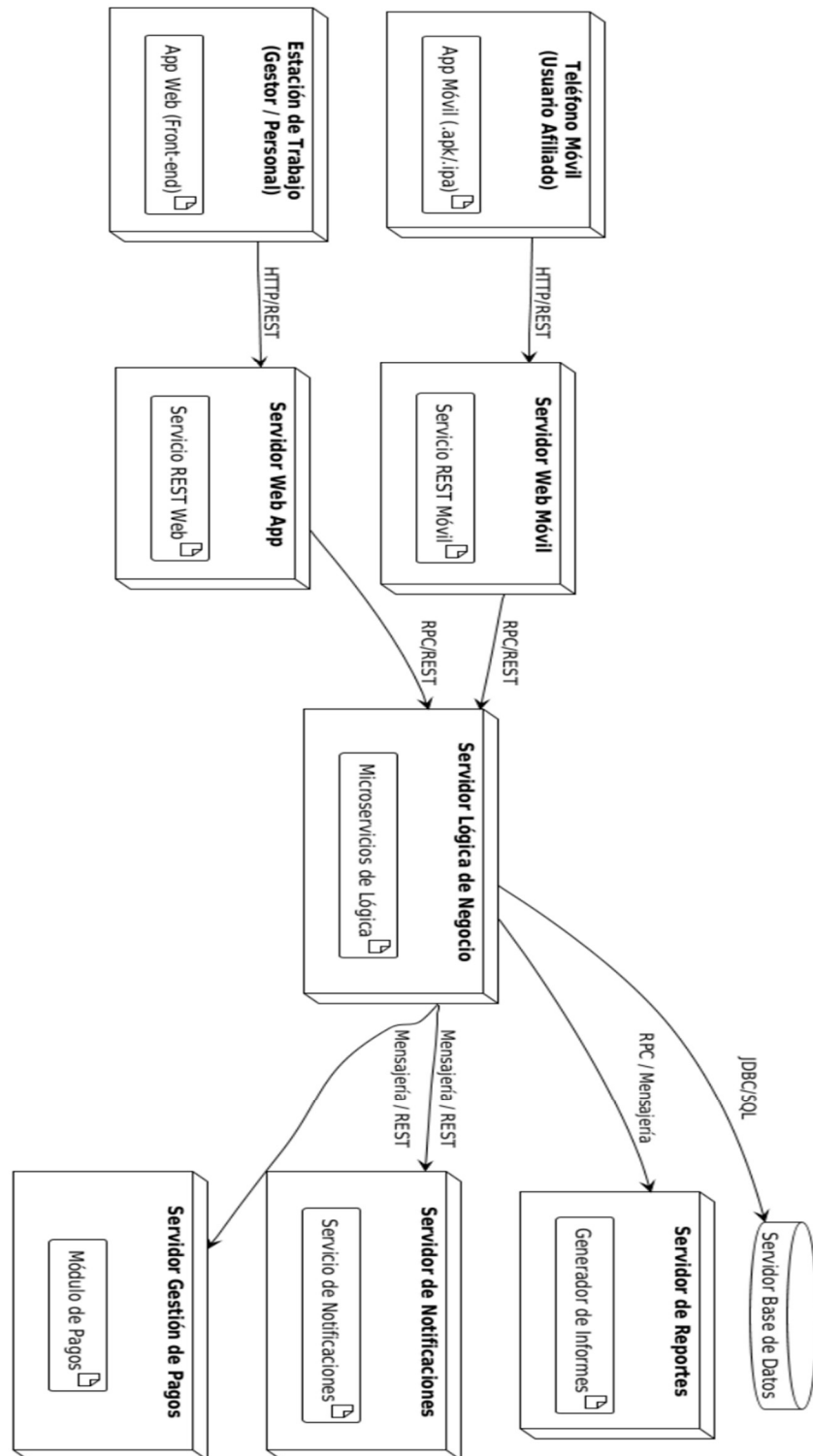


Figura 5. Diagrama de despliegue para el caso RedBiln

Creado en PlantUML (2025): <https://www.plantuml.com/>

Parte 3. MODELADO DE HISTORIAS DEL USUARIO PARA EL CASO DE ESTUDIO

Historia Usuario	COMO UN: <nombre de actor del sistema>	QUIERO: <funcionalidad>	CON EL FIN DE: <valor de la historia>	CRITERIOS DE ACEPTACIÓN: <criterios para validar la historia>
a. Registrar usuario/lector afiliado	Usuario Afiliado Bibliotecario	Afiliar a un usuario	Poder acceder a los servicios que ofrece RedBiln	Un formulario con los campos necesarios. Se envían los datos válidos y se responde con un correo de confirmación, en caso de confirmarse duplicado o falta de información muestra mensaje de error.
b. Registrar préstamo	Usuario Afiliado Bibliotecario	Realizar el préstamo de un libro a un usuario afiliado	Actualizar el sistema con el préstamo y establecer fecha devolución	Ingresa ID de usuario y código de libro, el sistema procede a verifica disponibilidad del libro y afiliación. Si el usuario tiene deudas o el libro no está disponible da error.
c. Registrar personal biblioteca	Gestor de RedBiln	Agregar personal autorizado en Biblioteca	Personal que pueda validar disponibilidad de libros y ayudar a usuarios a afiliarse	Formulario de credenciales para el rol de bibliotecario, se envía correo de confirmación al trabajador, en caso de faltar datos se indica error.
d. Registrar pago	Usuario afiliado Bibliotecario	Realizar el pago de la afiliación, multa o renovación.	Actualizar el estado de la deuda y obtener comprobante	Se selecciona el tipo de pago y monto. Permite funcionalidad para pagos en línea, como en ventanilla generando comprobante en ambos casos.
e. Generar notificaciones	Sistema (Automático)	Enviar notificaciones automáticas a usuarios	Alertar de eventos y recordatorio de pagos	Envía notificaciones, para conocer el estado del préstamo, si la fecha limite esta pronto a alcanzarse, aviso de confirmación de pago y recordatorio de renovación para mantenerse afiliado

Tabla 2. Historias de usuario

Parte 4. STORY MAPPING

La definición provista por Patton (2014) cuenta que “el *story mapping* de usuario es una herramienta valiosa para el desarrollo de software, una vez se comprende por qué y cómo usarlo. La cual puede ayudar a tu equipo a mantenerse enfocado en los usuarios y sus necesidades sin perderse en el entusiasmo por las características individuales del producto.” Según Carroll (2020), “Los story mapping son páginas web que combinan mapas con contenido multimedia para contar historias.” Dicha herramienta es usada comúnmente en metodologías ágiles como Scrum o Extreme, las cuales ocupan entender de forma rápida y precisa los sucesos que llevaron a darse para que se de la historia de usuario, la herramienta Story Mapping busca responder las preguntas: ¿Quiénes actuaran?, ¿Qué actividades se deben hacer?, ¿Qué tareas compone cada actividad?, definir prioridades y secuencias y ¿Cuándo deber salir estas tareas?

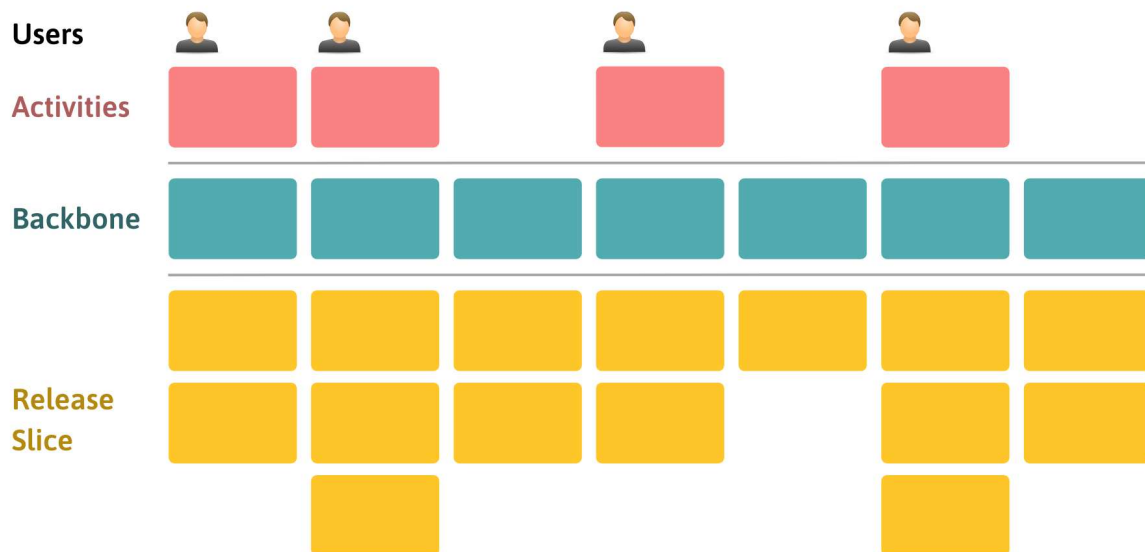


Figura 6. Ilustración “Story Mapping”. Extraído del sitio Web ProductGo (2020)

Parte 5. RIESGOS ASOCIADOS A LOS REQUERIMIENTOS DEL PROYECTO

Riesgo	Probabilidad	Impacto	Acciones de mitigación
1. Tener personal no capacitado	5% (Muy Baja)	Al tratarse de un proyecto novedoso en el país, es muy probable que los desarrolladores deban enfrentarse a nuevas problemáticas.	Previo a iniciar a escribir código, es necesario que los desarrolladores sepan del lenguaje a utilizar y capacitarse a las librerías que se usaran al diseñar.
2. Complejidad de las tareas	20% (Baja)	Al ser un proyecto grande como múltiples capas de desarrollo, puede suceder si no se da el tiempo necesario de planificación que se creen tareas que queden pendientes de subdividir.	Otorgar el tiempo adecuado a la planificación, realizar reuniones constante con el equipo de usuarios, de desarrollo y de inversión, con el fin de cuidar todo a detalle.
3. Falta en la seguridad de datos personales	10% (Baja)	Al no ser adecuado pertinentemente, puede ocasionar problemas legales, si se da un caso de filtración de datos sensibles.	La seguridad de los datos es prioridad 0, desde el inicio del proyecto hasta el fin del mismo, nada es lo suficiente barato y todo el tiempo es bien invertido, cuando se trata en cuidar los datos de los usuarios
4. Fallo en los sistemas de pagos por causas externas	99% (Casi seguro)	Esto impactaría tanto en pagos como virtual como al momento de ser presencial, es algo que sucede en algún momento de vida de la app	Contemplar la posibilidad de tener al menos 2 cuentas de bancos en diferentes entidades, por si alguna falla o preferencia del cliente.
5. Sincronización al actualizar datos	40% (Posible)	Al haber múltiples usuarios en busca de varios libros, podría suceder que 2 o más usuarios deseen el mismo y este sea escaso, generando falsos positivos, donde indique reserva asertiva, pero al rato deba negarse por falta del recurso.	Aplicar sistemas predictivos, sobre los productos más selectos y notificar a las bibliotecas, cuando alguna de sus obras este pronta agotarse y hacer pre reserva de libro antes de aprobar el préstamo, similar a sistemas de cine y matricula.

Tabla 3. Riesgos que pueden ocurrir durante el proyecto

Parte 6. PRINCIPIOS SOLID

Sección A. Definición

La comunidad de ingeniería de software ha formulado una serie de directrices que buscan orientar a los desarrolladores hacia la construcción de código de alta calidad. Entre las posibles se encuentran los principios SOLID emergen como un pilar fundamental. Acuñados por Robert C. Martin (también conocido como Uncle Bob) a principios de los 2000, y popularizados por Michael Feathers, estos principios no son meras sugerencias, sino axiomas de diseño orientados a objetos que, cuando se aplican adecuadamente, pueden transformar la forma en que se concibe y construye el software. El acrónimo SOLID engloba cinco principios interconectados: **Single** Responsibility Principle (Principio de Responsabilidad Única), **Open**/Closed Principle (Principio de Abierto/Cerrado), **Liskov** Substitution Principle (Principio de Sustitución de Liskov), **Interface** Segregation Principle (Principio de Segregación de Interfaces), y **Dependency** Inversion Principle (Principio de Inversión de Dependencias).

Sección B. Explicación de cada uno de los principios

El **Principio de Responsabilidad Única** establece que "una clase debe tener solo una razón para cambiar" (Martin, 2002). En otras palabras, cada clase, módulo o función dentro de un sistema debe ser responsable de una única y bien definida parte de la funcionalidad. La esencia de este principio radica en la encapsulación de las responsabilidades, de modo que, si se presenta un cambio en el sistema, este cambio solo afecte a una única clase o a un número mínimo de ellas.

El **Principio de Abierto/Cerrado** postula que "las entidades de software deben estar abiertas para extensión, pero cerradas para modificación" (Martin, 2002). Esto significa que el comportamiento de un módulo debe poder extenderse para adaptarse a nuevos requisitos sin necesidad de alterar su código fuente existente. La clave para implementar el principio reside en la abstracción y el polimorfismo. En lugar de codificar directamente las implementaciones, se debe programar contra interfaces o clases abstractas.

El **Principio de Sustitución de Liskov**, formulado por Barbara Liskov, establece que "los objetos en un programa deben ser reemplazables por instancias de sus subtipos sin

alterar la corrección de ese programa" (Liskov, 1988). Es decir, si una clase B hereda de una clase A, entonces B debe poder ser usada en cualquier lugar donde se espera una A sin que el programa se comporte de manera inesperada. Este principio va más allá de la mera herencia sintáctica y se enfoca en el comportamiento.

El **Principio de Segregación de Interfaces** afirma que "los clientes no deberían ser forzados a depender de interfaces que no utilizan" (Martin, 2002). Esto se traduce en la recomendación de crear interfaces más pequeñas y específicas, en lugar de interfaces grandes y monolíticas. Al crear interfaces se hacen con clases para métodos que quizás no necesiten, lo que conduce a código innecesario, confuso y propenso a errores.

El **Principio de Inversión de Dependencias** establece que: "los módulos de alto nivel no deben depender de módulos de bajo nivel. Ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones" (Martin, 2002). Este principio busca reducir el acoplamiento directo entre módulos, particularmente entre la lógica de negocio de alto nivel y las implementaciones de bajo nivel.

Parte 7. ESTANDAR DE CALIDAD

Sección A. ISO/IEC 25010:2023 – Modelo de calidad del producto software (SQuaRE), explicación

El estándar de calidad seleccionado es ISO/IEC 25010:2023 – Modelo de calidad del producto software (SQuaRE), la selección se debe a que es el estándar de calidad más reciente y actualizado que sirve para evaluar la calidad de productos de software e ICT. La cual se define por la búsqueda de la calidad en 9 aspectos esenciales: funcionalidad, rendimiento, seguridad, compatibilidad, usabilidad, fiabilidad, mantenibilidad, portabilidad y eficiencia, las cuales se pueden aplicar en el caso de estudio de RedBiln. Este estándar de calidad permite diseñar requisitos de calidad precisos, definir métricas y criterios de aceptación para evaluar artefactos, servicios REST, aplicaciones móviles/web y componentes de infraestructura. Al ser este reconocido internacionalmente provee una alta presentación mercado e inversionistas y siendo consistente durante el ciclo de vida que el proyecto posee.

Sección B. Características de dicho estándar de calidad

Ahora se nombrarán 3 características del estándar de calidad ISO/IEC 25010:2023:

- **Compatibilidad:** Esta evalúa la capacidad del software de coexistir e interoperar con otros sistemas, se logran con diferentes pruebas de interoperabilidad y conformidad de APIs.
- **Seguridad:** Esta evalúa la protección frente a accesos no autorizados y el manejo correcto de la información como datos sensibles y personales, se realiza con mediciones de tasa de vulnerabilidad, cobertura de cifrado y auditoria de accesos.
- **Satisfacción:** Esta evalúa el grado de cumplimiento de las expectativas y comodidad del usuario, las medidas se logran capturar con encuestas de satisfacción o Net Promoter Scope (NTS)

CONCLUSIÓN

Se ha llegado a la parte final del documento, en el cual se brindarán las conclusiones relacionado al objetivo del proyecto, el cual el ejecutor luego de hacer uso y responder las preguntas, adquirió los conocimientos durante el transcurso de todo el curso y a lo largo de los proyectos, lo cual ha sido enriquecedor y en el futuro se podrá recomendar a los encargados el uso de las metodologías aquí aplicadas.

La primera conclusión, que se llegó en este proyecto ha sido aprender sobre los patrones arquitectónicos al mencionarlos entender la forma como se definen, sus ventajas y desventajas con respecto a otras y se ilustraron para terminar de aclarar su composición y como cada una de estas es aplicable según el tipo de proyecto que se esté trabajando.

Continuando con el caso de estudio, se trabajó en las historias de usuario relacionando lo que estas necesitan para funcionar y como estas se validan para su confirmación, y por medio del diagrama de despliegue conocemos como los programas del sistema, interactúan con el hardware para aplicarse como un conjunto de soluciones que interactúan entre sí con el fin de ser amigable con los usuarios y la gestión de los procesos.

Toda propuesta posee riesgos y para el caso de estudio se trabajó sobre 5 de estos, en los cuales se pensó en posibles formas de que algo tomase más tiempo de trabajo en cumplir con los lineamientos del mismo, ya que pueden poseer ciertas consecuencias que retrasen al proyecto hasta posibles consecuencias legales si no se actúan con la suficiente madurez.

Los riesgos siempre se pueden mitigar de alguna u otra forma, uno de los mayores problemas es la necesidad de mantener un estándar entre todos los miembros, para ello muchos proyectos de software trabajan bajo los principios Solid, donde la construcción del programa debe buscar alinearse a estos.

Y finalmente, los estándares de calidad como el seleccionado ISO/IEC 25010:2023 (SQuaRE), provean a los equipos desarrolladores las métricas y medidas que se van a evaluar en busca de procedimientos que ayuden a mantener la solución con el mayor ciclo de vida posible y que las actualizaciones no sean el típico “si funciona no lo toques”.

REFERENCIAS

- AlexHost. (2024, octubre). *What is MVC: advantages and disadvantages of MVC*. AlexHost.
<https://alexhost.com/faq/what-is-mvc-advantages-and-disadvantages-of-mvc/>
- Carroll, A. (2020). Story maps. En D. Richardson, N. Castree, M. F. Goodchild, A. Kobayashi, W. Liu, & R. A. Marston (Eds.), *International encyclopedia of geography: People, the Earth, environment and technology*. Wiley. <https://doi.org/10.1002/9781118786352.wbieg2048>
- Graça, H. O. (2017, 3 de agosto). *Layered architecture diagram* [Imagen]. Recuperado de <https://herbertograca.com/2017/08/03/layered-architecture/>
- GeeksforGeeks. (2024, 4 de julio). *MVC Architecture* [Imagen]. Recuperado de <https://www.geeksforgeeks.org/system-design/mvc-architecture-system-design/>
- GeeksforGeeks. (2025, julio 11). *Client–Server model*. GeeksforGeeks.
<https://www.geeksforgeeks.org/system-design/client-server-model/>
- GeeksforGeeks. (2024, 16 de julio). *Client-server architecture* [Imagen]. Recuperado de <https://www.geeksforgeeks.org/system-design/client-server-architecture-system-design/>
- InterServer. (2016, octubre 1). *What is MVC? Advantages and disadvantages of MVC architecture*. InterServer. <https://www.interserver.net/tips/kb/mvc-advantages-disadvantages-mvc/>
- ISO. (2023-11). *ISO/IEC 25010:2023 – Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Standard.
- ISO25000.com. (s. f.). *ISO/IEC 25010: Product quality model definitions*. Descripción del modelo de calidad y características.
- Liskov, B. (1988). *Data abstraction and hierarchy*. ACM SIGPLAN Notices, 23(5), 17-34.
- Martin, R. C. (2002). *Agile software development: Principles, patterns, and practices*. Prentice Hall.
- Pantelon, G. & Rinaudo, L. (2015). *Ingeniería de Software*. 1ed. Buenos Aires, Argentina.
- Patton, J., & Economy, P. (2014). *User story mapping: Discover the whole story, build the right product*. O'Reilly Media. <https://www.oreilly.com/library/view/user-story-mapping/9781491904909/>
- ProductGo. (2022, abril 2022). *What is user story map? How it works in Jira*. Recuperado de <https://userstorymap.io/what-is-user-story-map/>

TechTarget. (2023). *The pros and cons of a layered architecture pattern*. TechTarget.

[https://www.techtarget.com/searchapparchitecture/tip/The-pros-and-cons-of-a-layered-architecture-](https://www.techtarget.com/searchapparchitecture/tip/The-pros-and-cons-of-a-layered-architecture-pattern)

pattern Kitrum. (2024, diciembre). *Client-server architecture – advantages and disadvantages*. Kitrum.

<https://kitrum.com/blog/client-server-architecture-advantages-and-disadvantages/>

Universidad Estatal a Distancia. (2025). Caso de estudio para proyectos: Red de Bibliotecas

Interconectadas (RedBiln) [*Manuscrito no publicado*]. Cátedra de Ingeniería de Software, Escuela de Ciencias Exactas y Naturales, Carrera Ingeniería Informática.

Yokwe, J. (2023, febrero). *Dissecting layered architecture*. DEV Community.

<https://dev.to/yokwejuste/dissecting-layered-architecture-2ppb>